

トラブルシューティングレポート：Slave Aのソフトウェアシリアル出力トラブル

1. 事象の概要

ADX Core-D LN-485 Test Firmware (Phase 5) において、`ROLE_SLAVE_A` として設定した ATtiny1616 マイコンから、シリアルモニタ（SoftwareSerial経由）に何も出力されない事象が発生した。Masterからのデータ送信は確実に行われている環境下でのトラブルであった。

2. 初期検証と切り分け

ハードウェア（USB-シリアル変換モジュール、配線等）の不具合を疑い、以下を実施した。* TX/RXのクロス接続確認 * GNDの共通化確認 * 通信速度設定（9600bps）およびマイコンのクロック設定の確認 * 最小構成の「Hello from ATtiny1616!」コードによる出力テスト

結果: 最小構成コードでは正常に出力されたため、**ハードウェアおよび接続設定には問題がないことが確定**した。

3. 原因の特定

コードの解析結果、受信処理ロジックに**デッドロック（フリーズ）を引き起こすバグ**が存在することが判明した。

根本原因の詳細

Slave Aの受信ポーリング処理は以下のように実装されていた。

```
while (USART0.STATUS & USART_RXCIF_bm) { // (1) 受信完了フラグを待つ
    // ...
    if (USART0.STATUS & USART_ISFIF_bm) { // (2) 同期エラーをクリアする
        // クリア処理
    }
}
```

ATtiny1616のLINオートボーレート（LINAUTO）機能の仕様上、ノイズ等により**同期エラー（ISFIF）が発生すると、受信完了フラグ（RXCIF）は絶対に立たない**。RS-485の半二重通信では待機中にノイズを拾いやすく、起動直後に高確率で ISFIF が発生する。しかし、上記のコードでは RXCIF が立たない限り ISFIF のクリア処理（2）へ到達できないため、一度エラーが発生すると永久に受信機能がロックされ、プログラムが沈黙（シリアル出力が出ない）状態に陥っていた。

4. 解決策

エラー検知とクリアの処理を、受信完了待ちの `while` ループの外側に移動させることで解決する。

修正コード (Slave A / Slave B 共通)

```
// 修正前
// 3. 【通信層】 高速ポーリング処理
// while (USART0.STATUS & USART_RXCIF_bm) { ... } の中にISFIFクリア処理があった

// 修正後
// 3. 【通信層】 高速ポーリング処理

// RXCIF(受信完了)を待つ前に、ISFIFエラーでロックされていないか常に監視・復帰する
if (USART0.STATUS & USART_ISFIF_bm) {
    pcSerial.println(F("[Slave Error] ISFIF Sync Error! Auto-recovered.));
    USART0.STATUS = USART_WFB_bm | USART_ISFIF_bm | USART_BDF_bm; // フラグクリア&再アーム
}

while (USART0.STATUS & USART_RXCIF_bm) {
    uint8_t rxHigh = USART0.RXDATAH; // ① 先にステータス取得
    uint8_t rxLow  = USART0.RXDATAH; // ② 次にデータ取得

    // パリティエラー・フレーミングエラーのチェック
    if (rxHigh & (USART_PERR_bm | USART_FERR_bm | USART_BUFOVF_bm)) {
        pcSerial.println(F("[Slave Error] Header PERR/FERR/OVF detected. Discarding.));
        USART0.STATUS = USART_WFB_bm | USART_ISFIF_bm | USART_BDF_bm;
        continue;
    }
    // 以降、元の受信処理
```

5. 推奨される追加対策

エラー発生 の 頻度を 下げる ための 予防策 として、Master側が送信するブ레이크信号のデリミタ（区切り）期間を延長することを推奨する。これにより、Slave側の同期マージンが広がり通信が安定する。

Master側の修正コード

```
// sendLinBreak() 関数内
PORTA.OUTSET = PIN1_bm;
delayMicroseconds(tBit * 2); // 1 Tbitから2 Tbitへ延長し、確実に同期させる
```